

# **RAPPORT DE PROJET**

## **MyTeams**

Développement d'une application de messagerie en réseau

**Étudiant :** BOKO Ariel

**Mail :** ariel.boko@epitech.eu

**Ville :** EPITECH Bénin

**Équipe :** BOKO Ariel, AZANMASSO Divine, GOUGBE Chris Maël

**Promotion :** 2ème année, Promotion 2029

**Date :** 24 mai 2026

---

# INTRODUCTION

Ce rapport porte sur le projet MyTeams, réalisé au cours du quatrième semestre de ma deuxième année à Epitech Bénin. Mené en trinôme avec Divine AZANMASSO et Chris Maël GOUGBE, ce projet consistait à concevoir et développer, de bout en bout, une application de messagerie collaborative fonctionnant en réseau. L'inspiration vient d'outils professionnels comme Microsoft Teams ou Slack, mais l'ensemble a été développé entièrement en langage C, sans bibliothèque externe de haut niveau.

Ce document revient sur les choix techniques que nous avons effectués, la manière dont le projet s'est déroulé concrètement, les obstacles rencontrés en cours de route, ainsi que les enseignements que j'en retire.

## 1. Présentation du projet

MyTeams est un projet de programmation réseau dont l'objectif est de permettre à plusieurs utilisateurs de communiquer en temps réel via un système de canaux organisés en équipes et en fils de discussion. Techniquement, il se décompose en deux programmes distincts : un **serveur** chargé de centraliser les connexions et de distribuer les messages, et un **client** en ligne de commande permettant à chaque utilisateur d'interagir avec ce serveur.

Les contraintes imposées par l'école étaient strictes : développement exclusivement en C, interdiction d'utiliser des bibliothèques externes de haut niveau, gestion manuelle de la mémoire, et protocole de communication conçu entièrement par l'équipe. C'est précisément ce cadre contraignant qui rend ce projet formateur, il oblige à comprendre en profondeur ce qui se passe sous la surface de tout système réseau.

## 2. Choix techniques et architecture

Pour mener à bien ce projet, nous avons mobilisé les outils fondamentaux de la programmation réseau en C. Le tableau suivant présente les principales briques techniques mises en œuvre :

| Technologie / Concept         | Rôle dans le projet                                                   |
|-------------------------------|-----------------------------------------------------------------------|
| <b>Sockets TCP</b>            | Canal de communication fiable entre le client et le serveur           |
| <b>bind / listen / accept</b> | Mise en écoute du serveur et acceptation des nouvelles connexions     |
| <b>send / recv</b>            | Envoi et réception des messages entre les deux parties                |
| <b>poll()</b>                 | Gestion simultanée de plusieurs clients sans bloquer le serveur       |
| <b>Protocole personnalisé</b> | Format de trame défini par l'équipe pour structurer tous les échanges |
| <b>Persistance en mémoire</b> | Stockage et organisation des équipes, canaux et utilisateurs          |

L'architecture repose sur un modèle **serveur central** : un seul processus gère l'ensemble des connexions grâce à **poll()**, qui permet de surveiller simultanément plusieurs descripteurs de fichiers sans créer un thread par client. Cette approche, sobre mais robuste, nous a contraints à concevoir soigneusement la structure des données dès la phase de conception.

## 3. Déroulement du projet

Le projet s'est étalé sur environ cinq semaines. La première semaine a été consacrée à la conception : définition de l'architecture globale, choix du format du protocole et répartition des rôles au sein de

l'équipe. J'ai pris en charge le développement du serveur, tandis que Divine AZANMASSO et Chris Maël GOUGBE s'occupaient respectivement du client et de la gestion des données.

La deuxième semaine a marqué le début de l'implémentation concrète. Nous avons d'abord mis en place les fonctions réseau de base, connexion d'un utilisateur, création d'équipes et de canaux, avant de nous attaquer à la transmission des messages. Cette phase a nécessité de nombreux échanges entre les membres de l'équipe pour garantir la cohérence entre le client et le serveur.

À mi-parcours, nous avons organisé plusieurs sessions de tests avec plusieurs clients connectés simultanément, ce qui a révélé les premiers problèmes de robustesse. La dernière phase a été particulièrement dense : correction des fuites mémoire à l'aide de **Valgrind**, gestion de la persistance des données et rédaction de la documentation technique. Certaines fonctionnalités n'ont pas pu être finalisées dans les délais, mais l'essentiel du système était opérationnel.

## 4. Difficultés rencontrées

Ce projet nous a confrontés à des problèmes concrets que les exercices habituels n'abordent pas. Voici les plus significatifs.

**La gestion des déconnexions inattendues :** Lorsqu'un client se déconnectait brutalement, sans envoyer de message de fermeture, le serveur tentait parfois d'écrire sur un descripteur de fichier invalide, provoquant une erreur de segmentation difficile à reproduire. Ce problème a été résolu en vérifiant systématiquement la valeur de retour de chaque appel à **recv()**, une discipline que j'ai conservée depuis.

**La cohérence des données partagées :** Avec plusieurs clients connectés en parallèle, assurer la cohérence des données s'est révélé plus complexe qu'anticipé. Certaines situations de compétition (**race conditions**) généraient des comportements erratiques et difficiles à isoler. Nous avons dû revoir la gestion de l'état global du serveur pour corriger ces cas méthodiquement.

**La conception du protocole de communication :** Élaborer un protocole maison s'est avéré plus exigeant que prévu. Des erreurs de sérialisation et de désérialisation ont engendré des corruptions de messages qui n'étaient pas immédiatement visibles. L'identification précise des trames défaillantes a nécessité le recours à des outils de débogage réseau.

## 5. Résultats et bilan

Le projet a été évalué à **70 %** lors de la soutenance. Les fonctionnalités essentielles étaient présentes et opérationnelles : connexion des utilisateurs, création d'équipes et de canaux, messagerie directe entre utilisateurs. En revanche, certaines fonctionnalités avancées, notamment la persistance complète des données entre les sessions et quelques commandes optionnelles, n'ont pas pu être finalisées dans le temps imparti.

Ce résultat reflète honnêtement l'état du projet à la date de la soutenance. Les 30 % manquants s'expliquent principalement par un manque d'anticipation sur la complexité de certaines parties du protocole, ainsi que par du temps perdu à corriger des problèmes qui auraient pu être évités avec une meilleure organisation dès le départ. Ces constats constituent des leçons concrètes que je retiens pour la suite.

## CONCLUSION

MyTeams a été l'un des projets les plus exigeants réalisés jusqu'ici à Epitech Bénin. Il m'a conduit à approfondir ma maîtrise de la programmation réseau en C, à travailler en équipe sur une base de code partagée, et à résoudre des problèmes techniques qui ne se règlent pas en quelques minutes de recherche.

Au-delà des aspects techniques, sockets, protocole TCP, gestion de la mémoire, multiplexage de connexions, ce projet m'a transmis une leçon plus fondamentale : la qualité d'une architecture se mesure

à la facilité avec laquelle on peut la faire évoluer. Une base mal conçue au départ engendre des coûts bien plus importants en fin de projet.

Pour les projets à venir, je compte adopter une démarche plus itérative : avancer par étapes courtes, tester régulièrement chaque fonctionnalité et ne pas chercher à tout implémenter simultanément. C'est certainement l'enseignement le plus durable que ce projet m'ait apporté.

---

*Fait à Cotonou, le 24 mai 2026*

**BOKO Ariel**, Étudiant Epitech Bénin, 2ème année